
Sovereign retrieval for Hacker News stories

Search HN
Personal research project

Abstract

We study locally served embedding models and hybrid retrieval for a Hacker News search agent. Using 196 questions over 98 target stories, we compare four open-weight embedding models with a hosted text-embedding-3-large baseline, evaluate three finalists in an agent loop, and measure the accuracy and cost of serving the selected model. Perplexity’s 0.6B model provides competitive retrieval quality and embeds a 64,638-story corpus in 225 seconds using BF16 on an RTX 3060. Adding a small BM25 contribution raises exact top-20 target recovery from 153 to 158 questions. PostgreSQL HNSW reduces median warm vector lookup from 165 to 13 milliseconds in a matched CPU experiment, recovering 157 of those 158 targets. The results support a compact retrieval system built from Perplexity BF16 embeddings, PostgreSQL BM25, and HNSW.

1. Introduction

Finding a remembered Hacker News story often requires connecting a description to a short title or URL. Lexical search provides strong signals for names and distinctive phrases, while embeddings can bridge paraphrases and vocabulary differences. For an LLM search agent, retrieval quality also affects how many searches and model turns are needed before it can inspect the relevant discussion.

This project seeks a retrieval system whose embedding weights and inference can be operated locally, while allowing the agent’s driver model to remain hosted. The practical constraints are modest hardware, a corpus admitted by vote count, and an existing PostgreSQL deployment. We investigate three questions: which small embedding model offers a useful quality–cost balance; how much lexical retrieval should contribute; and how much accuracy approximate nearest-neighbor search sacrifices for lower query latency.

The study proceeds from an initial lexical and database comparison to a controlled embedding-model bakeoff, followed by serving and index experiments on the selected model. The resulting recommendation is Perplexity’s 0.6B embedding model in BF16 at 1024 dimensions, combined with a lightly weighted BM25 branch and a high-recall HNSW configuration. This paper presents the evaluation protocol, the evidence for that selection, and the remaining validation work.

2. Methods

2.1. Corpus and questions

Documents consist of a story title, newline, and URL. Eligible stories are neither dead nor deleted; self-posts are included. The initial engine experiments used a two-year corpus with score at least 10. The later model comparison and tuning experiments used the same date window with a score floor of 25:

Experimental stage	Score floor	Documents
Initial engine comparison	10	105,081
Model comparison and tuning	25	64,638

The window is 2024-09-04 through 2026-09-04. Corpus snapshots and query strings are frozen and identified by hashes throughout each stage.

The evaluation contains 196 synthetic questions over 98 stories, with one entity-oriented and one paraphrased question per target. Luna generated the questions from frozen story and comment snippets, followed by a separate review for source support and identifying-information leakage. The set includes 120 questions in the recent cohort and 76 in the older cohort. Retrieval indexes title and URL information; the agent can subsequently read comments to answer the question.

2.2. Retrieval and embedding models

Exact cosine search provides the dense reference ranking. The lexical branch uses the title-only pg_textsearch BM25 index in PostgreSQL. Hybrid retrieval combines the first 100 results from each branch using reciprocal-rank fusion:

$$s(d) = \frac{1}{60 + r_D(d)} + \frac{w_L}{60 + r_L(d)}. \quad (1)$$

Each term contributes only when the document occurs in that branch’s candidate list. Dense weight is fixed at 1 and lexical weight is varied. Weight zero returns the dense list. Ties are resolved by story ID after candidate selection.

We compare Perplexity pplx-embed-v1-0.6b, Qwen3-Embedding-0.6B, Jina’s premerged v5 text-small retrieval model, and Nemotron-3-Embed-1B-BF16. The hosted control is text-embedding-3-large (TE3). Candidate selection considered English retrieval benchmarks, model size, publisher history, licensing, and local serving support.

Each model uses its documented query/document formatting and pooling. Qwen applies a query instruction and last-token pooling. Jina uses Query: and Document: prefixes with last-token pooling. Nemotron uses query: and passage: prefixes with bidirectional mean pooling. Perplexity uses mean pooling without role prefixes, followed by its published int8 output transform. Reference comparisons and singleton/batch checks precede full backfills. Dimension sweeps truncate cached vectors to a prefix and L2-normalize both documents and queries before scoring.

2.3. Serving and index experiments

Embedding backfills run on an RTX 3060 with 12 GiB VRAM. The initial Perplexity run uses FP32 in TEI 1.9.3; the subsequent BF16 experiment uses stock vLLM 0.28.0 with native Qwen3 bidirectional attention and mean pooling. Its client transforms pooled outputs with float32 tanh, multiplication by 127, rounding, and int8 clamping before cosine normalization.

The BF16 service permits 64 sequences and 8192 batched tokens, with a 2048-token input limit, Flash Attention, eager execution, and zero KV cache. Throughput includes HTTP transfer and durable shard writes. GPU memory is sampled every 500 ms. A pilot varies batch size before the full document pass.

Approximate retrieval uses pgvector HNSW with m=16 and ef_construction=128. We vary ef_search from 100 to 1000 and compare returned candidates with exact PostgreSQL top-100 lists. Query plans verify that the intended index or exact scan executes.

CPU latency is measured on the inference VM, which reports a Ryzen 5 5600X under KVM with eight exposed vCPUs and approximately 23 GiB RAM. PostgreSQL 17 and pgvector 0.8.2 run in a temporary container with a 10 GiB memory limit and 2 GiB shared buffers. Vectors are normalized float32 values stored using PostgreSQL’s external storage policy. Two randomized warm passes over all questions yield 392 measurements per method at concurrency one. The timer covers vector-query execution and fetching 100 IDs over VM loopback. The database timer excludes query embedding and lexical processing.

2.4. Evaluation measures

Each question has one labeled target. Recall@k is the fraction whose target appears in the first k results; nDCG@k discounts that target’s rank. We report cutoffs 8 and 20. ANN candidate recovery measures set overlap with exact top-k neighbors, which complements target recall by measuring how faithfully the index preserves the reference ranking.

In agent experiments, target exposure means the story entered the model’s consumed context. Evidence citation records whether the final answer cited target evidence. Model turns, search lists, token use, and billed driver cost characterize resource use. All questions remain in the denominator, including budget-exhausted sessions. Paired bootstrap analyses resample target stories, keeping each pair of question variants together.

3. Evaluation

3.1. Initial retrieval and engine comparison

The original Luna agent using untuned PostgreSQL FTS exposed the target in 150/196 cases. Subsequent TE3 dense and hybrid treatments each reached 182/196, with approximately 20% fewer model turns and 41–42% fewer search lists. Input-token use fell by 2.6% for dense and 7.5% for hybrid. This comparison measures the combined retrieval-interface change: corpus scope, pagination, and API-failure recovery also differed between the original and later runs.

Exact dense results agreed between PostgreSQL and DuckDB. In the lexical branch, replacing stock PostgreSQL text ranking with pg_textsearch raised Recall@8 from 45.9% to 61.2% on the initial slice; DuckDB BM25 reached 60.2%. PostgreSQL’s title-only index was retained for subsequent experiments. A Qwen3-0.6B reranker improved dense retrieval at rerank depth 30, but produced no net improvement in PG hybrid Recall@8. This favored continuing with dense and BM25 primitives.

3.2. Embedding quality and agent performance

Table 1 compares the native model recipes on the 64,638-story corpus. Hybrid scores in this comparison use lexical weight 0.5. Nemotron has the strongest native dense nDCG@20, while Perplexity matches its target count with half as many dimensions. TE3 remains competitive.

Table 1: Static model comparison. Target counts are out of 196; hybrid lexical weight is 0.5.

Model / compute	Dims	Dense hits@20	nDCG@20	Hybrid hits@20
Pplx / FP32	1024	153	.6002	157
Qwen / FP16	1024	149	.5815	156
Jina retrieval / FP16	1024	148	.5678	156
Nemotron / BF16	2048	153	.6154	159
Nemotron / shortened	1024	153	.5976	157
TE3-large	1536	154	.5873	160

Jina’s local results did not offset the additional license friction for this project. Perplexity, Qwen, and Nemotron advanced to the agent comparison. The driver was Luna through OpenRouter’s OpenAI route, using the same interface, exact dense retrieval, and ten-turn budget across the three arms. Perplexity used FP32 in these agent runs.

Table 2: Agent outcomes. TE3 is an earlier direct-API run and serves as historical context.

Dense provider	Exposure /196	Evidence cited /196	Mean turns
Pplx 1024 FP32	181	168	4.08
Qwen 1024 FP16	178	167	4.13
Nemotron 2048 BF16	184	168	4.03
TE3 1536, historical	182	172	3.95

The 588 finalist sessions produced 564 completed answers and 24 turn-budget exhaustions, with zero infrastructure failures. Their 2,399 streamed model requests cost \$3.3401. Execution took 5m43s at shared concurrency 16, excluding setup. Nemotron gained five exposures and lost two relative to Perplexity; the target-cluster bootstrap interval for that difference included zero. Both cited target evidence in 168 cases. These results made serving efficiency consequential to the final model choice.

3.3. Perplexity BF16 efficiency

An FP32 vLLM control first established compatibility with the TEI reference. Minimum cosine agreement was .999911 across all 196 query vectors and .999712 across 320 sampled/stress documents. The corresponding BF16 minima were .999336 and .997485. Full-corpus scoring retained 153 dense top-20 hits; top-8 hits changed from 136 to 135, while nDCG@20 increased slightly from .6002 to .6015.

BF16 reduced the complete Perplexity backfill from 14m05s to **3m45s**, averaging **287 stories/s** at batch size 64. Sampled peak GPU memory was **1,735 MiB**. For comparison, the tested Qwen FP16 recipe completed in 6m36s, Jina FP16 in 6m43s, and Nemotron BF16 in 15m06s. These measurements include the respective serving backends and serialization paths. The result supports Perplexity BF16 as the most attractive measured quality-throughput combination for this workload.

3.4. Dimension and hybrid-weight tuning

Perplexity’s BF16 dimension sweep favored retaining 1024 dimensions. At 768, dense top-20 hits remained 153, but nDCG@20 declined from .6015 to .5947 and top-8 hits from 135 to 133. At 512 and 256, top-20 hits fell to 145 and 130. Shortening therefore offered a storage tradeoff without improving inference throughput.

With 1024 dimensions fixed, a light lexical contribution improved retrieval:

Table 3: Exact BF16 retrieval with dense weight 1, RRF constant 60, and 100 candidates per branch.

Lexical weight	Hits@8 /196	Hits@20 /196	nDCG@20
0 (dense)	135	153	.6015
0.125	143	158	.6215
0.25	147	157	.6092
0.5	144	157	.6099
1	136	154	.6075
2	133	140	.5947
4	132	139	.5889
Lexical only	124	135	.5533

Weight 0.125 added five top-20 targets over dense without losing any; at top 8 it gained ten and lost two. Both question styles improved in nDCG@20. Weight 0.25 gave the highest top-8 target count, while larger lexical weights increasingly hurt paraphrase retrieval. We select 0.125 as the balanced setting and retain 0.25 as an alternative for a top-8-focused interface.

3.5. HNSW accuracy and latency

The full mirror contains 776,244, 509,107, and 353,173 eligible stories at score cutoffs 10, 25, and 50, respectively. At cutoff 25, this is 7.9 times the experimental slice. This scale motivated measuring approximate retrieval even though the evaluation corpus remained fixed.

On the original HNSW graph, increasing ef_search from 100 to 800 raised mean exact top-100 recovery from 85.93% to 98.85%, and hybrid top-20 target hits from 142 to 155. Exact hybrid search recovered 158 targets. At 1000, mean neighbor recovery reached 99.14% and hybrid hits reached 156.

The VM latency experiment found that PostgreSQL’s ordinary planner selected a serial exact scan even when two workers were allowed. Forcing a two-worker plan plus the leader reduced median

latency from 159.2 to 65.6 ms. HNSW 800 took 11.6 ms median on that run. A matched follow-up compared 800 and 1000 on a newly built VM graph:

Table 4: Matched warm VM measurements on the two-year slice. Latency covers the vector branch; hybrid accuracy is scored from its returned candidates.

Path	Median ms	p95 ms	Top-100 recovery	Hybrid hits@20
Exact serial	165.35	170.66	100%	158
HNSW 800	11.49	14.14	98.87%	155
HNSW 1000	13.43	17.16	99.16%	157

At 1000, median lookup was 12.3 times faster than serial exact and 17% slower than HNSW 800. The additional search effort recovered two more hybrid targets on this graph. The original graph recovered one more, showing modest build-to-build variation in the residual error. These measurements favor 1000 when accuracy is the priority.

3.6. Limitations

The evaluation has one labeled target per question and has been reused during tuning. A review of 18 residual misses found weak title/URL identification in ten and reasonable alternative answers in seven. Fresh natural queries and broader relevance judgments would strengthen the evidence. Historical FTS and TE3 comparisons also include interface or operational differences, whereas the contemporaneous finalist arms share a common protocol.

The agent comparison used exact dense retrieval and Perplexity FP32. BF16, hybrid weighting, and HNSW were evaluated subsequently through static tests. The selected combination still requires an integrated agent evaluation. ANN experiments cover the two-year slice without additional selective filters; full-corpus behavior, filtered candidate recovery, and concurrent load remain the principal deployment checks.

4. Conclusion

The experiments support a locally operated retrieval system using **Perplexity 0.6B BF16 embeddings at 1024 dimensions, PostgreSQL title-only BM25 with lexical RRF weight 0.125, and HNSW with ef_search=1000**. Perplexity delivered competitive agentic quality, a 225-second corpus backfill, and a modest GPU footprint. Light lexical fusion improved static target recovery, while HNSW provided a substantial CPU latency reduction with a small remaining accuracy cost.

The next stage is implementation and validation at deployment scale: preserve an exact reference path, test vote/date/domain filters, measure full-corpus load, and evaluate the combined recipe in the agent. The frozen evaluation, rollouts, recipes, and detailed experiment records are archived in Garage for that work.

References and reproducibility

The accompanying research notes 00–25 and Garage release `research-20260906-v4` contain per-case results, frozen inputs, and serving recipes. Artifact restoration is documented in `packages/search-research/docs/01-artifacts.md`.

1. Perplexity embedding model and output transform.
2. Qwen3-Embedding-0.6B Jina text-small retrieval Nemotron-3-Embed-1B-BF16.
3. vLLM 0.28.0 Qwen3 implementation.
4. pgvector 0.8.2.